Permisos y delegación de permisos en sistemas de ficheros Linux Ubuntu

INTRODUCCIÓN

La seguridad de un sistema operativo depende en gran medida de cómo se gestionan los permisos de acceso a los ficheros y directorios. Un sistema de permisos bien configurado impide que usuarios no autorizados lean, modifiquen o ejecuten fichero que no les corresponde, protegiendo asi tanto los datos del sistema como los de otros usuarios.

En esta practica estudiaremos en profundidad el siste de permisos en LINUX UBUNTU (que es el que más conozco y más configurable es) ya que es de los más robustos, tanto permisos efectivos, como se calculan, como se peuden delegar y finalmente los aplciaremos en un caso real.

1. ¿Qué son los permisos efectivos en sistemas de ficheros?
2. ¿Cómo se delegan permisos en sistemas operativos?
3. Caso de uso practico - mejor de seguridad de permisos
4. Práctica en UBUNTU - asignación ed permisos efectivos paso a paso

1 ¿ Que son los permisos efectivos en sistemas de ficheros?

Un permiso efectivo es el conjunto real de acciones que un usuario concreto puede realizar sobre un fichero o firectorio en un momento dado, teniendo en cuenta todos los permisos que le afectan: los asignados directamente a su usario , los de los grupos a los que pertenece y las restricciones del sistema.

La palabra clave es efectivo, no basta con saber qué permisos tiene un fichero de forma aislada, hay que calcular qué puede ahcer realmente el usuario que intenta acceder.

1.1 El modelo de permisos en Linux: UGO (USER, GROUP, OTHERS)

Linux usa un modelo de permisos basado en tres categorías, que se conocen con las siglas UGO

Categoría	A quien aplica
User (u)	El propietario del fichero, es el usuario que creó el fichero o al que se le asignó con chown.
Gruop (g)	Los miembros del grupo propietario del fichero. Un usuario puede pertenecer a varios grupos.
Others (o)	Cualquier usuario del sistema que no sea el propietario ni mienbro del grupo propietario.

Cada una de estas categorías tiene tres posibles permisos básicos:

Permiso	Símbolo	Valor numérico	Sobre fichero	Sobre directorio
Lectura	r	4	Ver el contenido del fichero	Listas los ficheros del directorio
Escritura	w	2	Modificar o eliminar el fichero	Crear, renombrar o eliminar ficehros dentro del directorio
Ejecución	x	1	Ejecutar el fichero como un programa	Entrar al directorio (cd) y acceder a su contenido
Sin permiso	-	0	Acción denegana	Acción denegada

Estos permisos se representan como una cadena de 9 caracteres. por ejemplo la cadena rwxr-xr-- se lee así, (se separa en tres bloques de tres simbolos y se suma su valor numérico):

r w x	r - x	r - -
Propietario (u)	Grupo (g)	Otros (o)
Lectura + Escritura + Ejecución (7)	Lentura + Ejecución (5)	Solo lectura (4)

1.2 Permisos especiales: SetUID, SetGID y Sticky Bit

Además de los permisos básicos rwx, Linux dispone de tres permisos especiales que modifican el comportamiento estándar.

Permiso especial	Descripción y uso
SetUID (s en user)	Cuando se ejecuta el fichero, el proceso adquiere los privilegiso del propietario del fichero, no del usuario que lo ejecutó.
SetGID (s en group)	En ficheros: el proceso se ejecuta con los permisos del grupo propietario. En directorios: los nevos ficheros creados dentro eherean el grupo del directorio, lo que facilita el trabajo colaborativo.
Sticky Bit (t en others)	En directorios: aunque varios usuarios puedan escribir en él, cada uno solo puede eliminar sus propios ficheros.

Info Extra

Estos permisos especiales son muy importantes en seguridad. un SetUID mal configurado en un binario puede convertirse en una vía de escalada de privilagios (privilege escalation)

1.3 Cómo se calculan los permisos efectivos

El kernel de Linux sigue un algoritmo preciso para determinar quñe permiso efectivo tiene un usuario cobre un fichero:

1º	¿Es root? UID 0 - Acceso total, ignorando todos los permisos excepto ejecución en algunos casos.
2º	¿El UID del usuario conincide con el propietario del fichero? - Se aplican los permisos de User (u).
3º	¿Alguno de los GID del usuario coincide con el grupo propietario? - Se aplican los permisos de Group (g).
4º	Si no coincide ni como propietario ni como grupo - Se aplican los permisos de Others (o).

1.4 umask: el filtro de permisos por defecto

Cuando se crea un fichero nuevo, no se crean con todos los permisos disponibles sino con los permisos máximos menos la umask. La umask es una máscara que ‘quita’ ciertos permisos de forma predeterminada.

El valor umask más común en Ubuntu es 022 lo que significa que:

Tipo	Permisos máximos	umaks	Permisos resultantes
Fichero nuevo	666 (r w - r w - r w -)	022(- - - - w - - w -)	664(r w - r - - r - -)
Directorio nuevo	777(r w x r w x r w x)	022(- - - w - - w - -)	775(r w x r - x r - x)

Comandos

Ver la umask actual:

$ umask

Ver en formato simbólico:

$ umask -S

2 Delegación de permisos en sistemas operativos

Delegar permisos significa transerir o comparir la capacidad de acceder o gestionar recursos del sistema a otros usuarios o procesos, de forma controlada sin necesidad de darles privileios de administrador completos (root). Una buena delegaciñon sigue el principio del minimo privilegio: cada usuario o proceso recibe únicamente los permisos que necesita para realizar su tarea y nada más.

2.1 Herramientas de delegación en Linux Ubuntu

a) sudo y sudoers: delegación de comandos especificos

sudo (Super User DO) permite que usuarios no privilegiados ejecuten comandos especificos con los privilegios de root u otro usuario, sin necesidad ed conocer la contraseña de root.

b) Grupos de usuarios: delegación por pertenencia a grupos

Los grupos son el mecanismo más natural de delegación en Linux. En lugar de asignar permisos a usuarios individuales, se asignan al grupo y luego se añaden usuarios al grupo. Si un usuario necesita acceder a un recurso, basta con añadirle al grupo correspondiente.

c) chown y chgrp: cambiar propietario y grupo

Los comandos chown (change owner) y chgrp (change group) permiten reasignar la propeidad de dicheros y directoris diferentes a usuarios o grupos, siendo una forma directa de delegación.

d) ACLs (Access Control Lists) : delegación granular

Las ACLs son una extensión del sistema de permisos clásico UGO que permiten asignar permisos a usuarios o grupos adicionales sobre un fichero, sin necesidad de cambiar su porpietario ni su nombre, Es la herramienta más precisa para la delegación de permisos en Linux.

2.2 Principios fundamentales de la delegación segura

Principio	Descripción
Minimo privilegio	cada usuario o proceso recibe solo los permisos imprescindibles para su funcion. Ninguno más.
Separación de roles	Las tareas críticas deben requerir la colaboración de al menos dos personas, evitando que una sola concertre demasiado poder.
Auditoria y trazabilidad	toda acción privilegiada debe quedar registrada (logs) para poder rastrear quién hizo qué y cuando.
Temporalidad	Los permisos especiales deben ser temporales cuando sea posible. Revocarlos cuando ya no sean necesarios.
Revisión periódica	Los permisos debes revisarse regularmente para eliminar accesos obsoletos o innecesarios

3 Caso de uso: Mejora de seguridad mediante permisos

3.1 Escenario: servidor web compartido en una empresa

Imaginemos una emprea con la siguiente estructura de personal

Usuario/Grupo	Rol y necesidad de acceso
www-data	Usuario del servidor web Apache. Necesita leer los ficheros web para servidores.
Desarrolladores	Grupo de desarrolladores que deben poder crear, editar y desplegar ficheros web.
Auditores	Grupo de auditoría que solo necesita leer los logs del servidor para informes.
root	Administrar el sistema, acceso total.
Verónica	Directora de marketing que solo puede subir imágenes a la carpeta /var/www/media.

Objetivo: configurar los permisos de froma que cada usuario solo pueda acceder a lo que necesita y nada más siguiendo el principio del minimo privilegio.

3.2 Estructura de directorios y permisos aplicados

La estructura del directorio web sería:

```
/var/www/proyecto/          .
|--- html/                  <--- ficheros públicos del sitio web
|---config/                 <--- ficheros de configuración ( cosas sensibles )
|---logs/                   <--- registros del servidor
|---scripts/                <--- scripts de mantenimiento ejecutables
|---media/                  <--- imágenes y archivos subidos
```

Directorio/Fichero	Propietario	Grupo	Permisos	Justificación
/var/www/proyecto	root	desarroladores	rwxrwxr-x(775)	Desarroladores gestionan todo, otros solo pueden leer ejecutar
html/	root	desarroladores	rwxrwsr-x(2 775)	SetGID: ficheros nuevos heredan el grupo 'desarroladores'
config/	root	root	rwx---------(700)	Solo root puede acceder. Contiene datos sensibles (Ej. contarseñas BD)
logs/	root	auditores	rwxr-x---(750)	auditores pueden leer logs, Otros sin acceso.
scripts/	root	desarrolladores	rwxr-x---(750)	Solo desarrolladores pueden ejecutar scripts de mantenimiento
media/	root	desarolladores	rwxrwxrwt(1777)	Sticky bit, cualqueir peude subir pero solo borra lo suyo

4 Práctica en UBUNTU: Asignación de permisos efectivos

A continuación llevamos a cabo la configurción descrita en el caso de uso anterior en un sistema Ubuntu real (Maquina virtual). Todos los comandos se ejecuntan en la terminal (bash) con privilegios de administrador.

Paso 1 - Crear los usuarios y grupos necesarios

Lo primero es crear los gripos y usuarios que intervienen en el escenario

```
# Crear los grupos:
$ sudo groupadd desarrolladores
$ sudo groupadd auditores

# Crear los usuarios
$ sudo useradd -m -s /bin/bash dev1
$ sudo useradd -m -s /bin/bash dev2
$ sudo useradd -m -s /bin/bash auditor1
$ sudo useradd -m -s /bin/bash veronica

# Asignar contraseñas a cada usuario
$ sudo passwd dev1
$ sudo passwd dev2
$ sudo passwd auditor1
$ sudo passwd veronica

# Añadir usuarios a sus grupos correspondientes
$ sudo usermod -aG desarrolladores dev1
$ sudo usermod -aG desarrolladores dev2
$ sudo usermod -aG auditores auditor1

# Verificar que los usuarios están en los grupos correctos
$ groups dev1
$ groups auditor1
```

```
vboxuser@Ubuntu:~$ sudo groupadd desarroladores
[sudo] password for vboxuser:
vboxuser@Ubuntu:~$ sudo groupadd auditores
vboxuser@Ubuntu:~$ sudo useradd -m -s /bin/bash dev1
vboxuser@Ubuntu:~$ sudo useradd -m -s /bin/bash dev2
vboxuser@Ubuntu:~$ sudo useradd -m -s /bin/bash auditor1
vboxuser@Ubuntu:~$ sudo useradd -m -s /bin/bash veronica
vboxuser@Ubuntu:~$ sudo passwd dev1
New password:
Retype new password:
passwd: password updated successfully
vboxuser@Ubuntu:~$ sudo passwd dev2
New password:
Retype new password:
passwd: password updated successfully
vboxuser@Ubuntu:~$ sudo passwd auditor1
New password:
Retype new password:
passwd: password updated successfully
vboxuser@Ubuntu:~$ sudo passwd veronica
New password:
Retype new password:
passwd: password updated successfully
vboxuser@Ubuntu:~$ sudo usermod -aG desarroladores dev1
vboxuser@Ubuntu:~$ sudo usermod -aG desarroladores dev2
vboxuser@Ubuntu:~$ sudo ussrmod -aG auditores auditor1
sudo: ussrmod: command not found
vboxuser@Ubuntu:~$ sudo usermod -aG auditores auditor1
vboxuser@Ubuntu:~$ groups dev1
dev1 : dev1 desarroladores
vboxuser@Ubuntu:~$ groupd auditor1
Command 'groupd' not found, did you mean:
  command 'groups' from deb coreutils (9.4-3ubuntu6.1)
Try: sudo apt install <deb name>
vboxuser@Ubuntu:~$ groups auditor1
auditor1 : auditor1 auditores
vboxuser@Ubuntu:~$
```

Paso 2 - Crear la estructura de directorios del proyecto

Creamos el árbol de directorios según el diseño del caso de uso:

```
# Crear el directorio raiz del proyecto
$ sudo mkdir -p /var/www/proyecto

# Crear los subdirectorios
$ sudo mkdir /var/www/proyecto/html
$ sudo mkdir /var/www/proyecto/config
$ sudo mkdir /var/www/proyecto/logs
$ sudo mkdir /var/www/proyecto/scripts
$ sudo mkdir /var/www/proyecto/media

# Crear algunos ficheros de prueba para las comprobaciones
$ sudo touch /var/www/proyecto/html/index.html
$ sudo touch /var/www/proyecto/config/db_config.php
$ sudo touch /var/www/proyecto/logs/access.log
$ sudo touch /var/www/proyecto/scripts/backup.sh

# Ver la estructura creada
$ tree /var/www/proyecto
# ( si no tienes instalado tree puedes hacerlo con “sudo apt inatall tree” )
```

Ubuntu (Instantánea 1) [Corriendo] - Oracle VirtualBox

Archivo Máquina Ver Entrada Dispositivos Ayuda

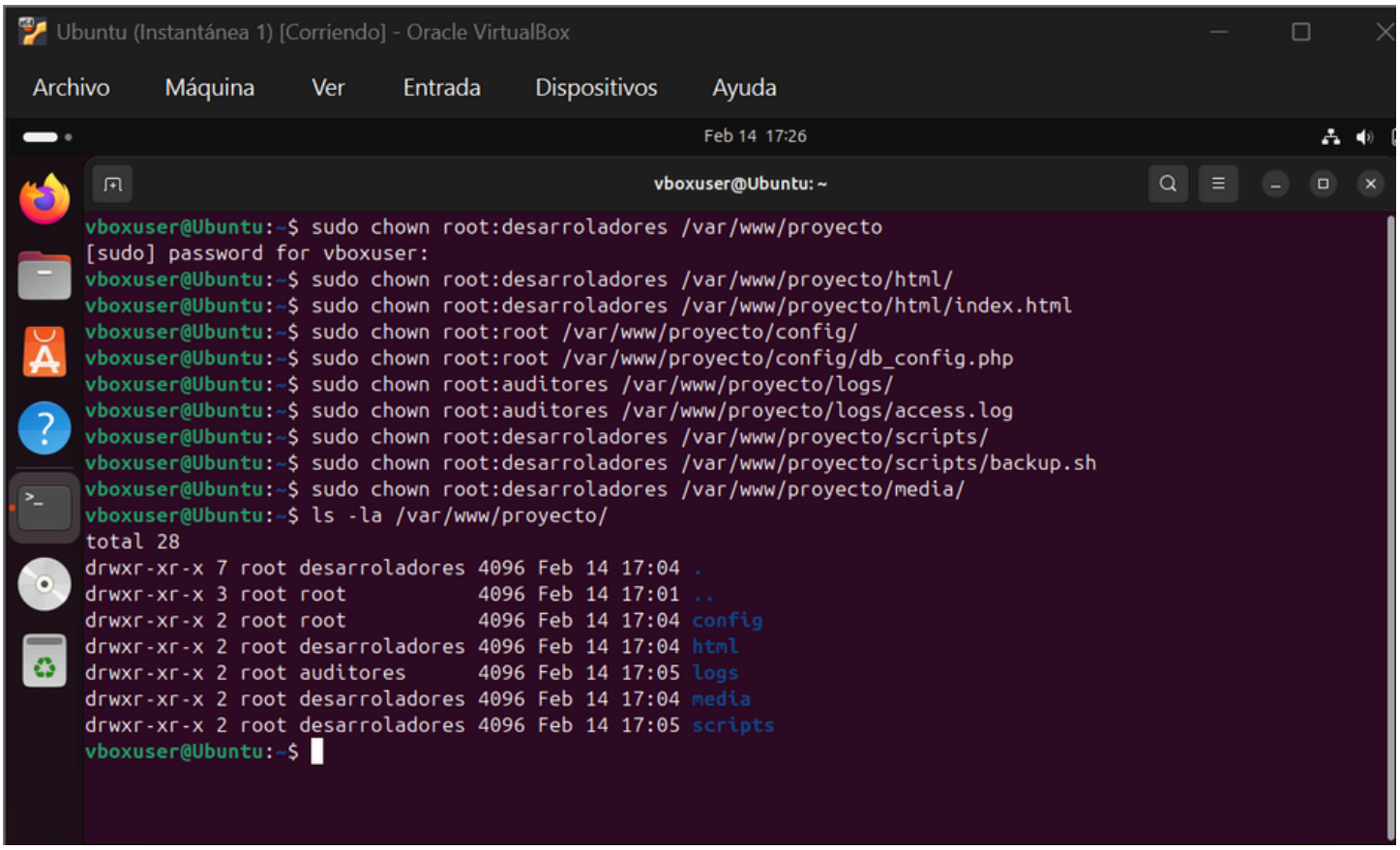
vboxuser@Ubuntu: ~

```
auditor1 : auditor1 auditores
vboxuser@Ubuntu:~$
vboxuser@Ubuntu:~$ sudo mkdir -p /var/www/proyecto
vboxuser@Ubuntu:~$ sudo mkdir /var/www/proyecto/html
vboxuser@Ubuntu:~$ sudo mkdir /var/www/proyecto/config
vboxuser@Ubuntu:~$ sudo mkdir /var/www/proyecto/logs
vboxuser@Ubuntu:~$ sudo mkdir /var/www/proyecto/scripts
vboxuser@Ubuntu:~$ sudo mkdir /var/www/proyecto/media
vboxuser@Ubuntu:~$ sudo touch /var/www/proyecto/html/index.html
vboxuser@Ubuntu:~$ sudo touch /var/www/proyecto/config/db_config.php
vboxuser@Ubuntu:~$ sudo touch /var/www/proyecto/logs/access.log
vboxuser@Ubuntu:~$ sudo touch /var/www/proyecto/scripts/backup.sh
vboxuser@Ubuntu:~$ tree /var/www/proyecto
Command 'tree' not found, but can be installed with:
sudo snap install tree  # version 2.1.3+pkg-5852, or
sudo apt  install tree  # version 2.1.1-2ubuntu3.24.04.2
See 'snap info tree' for additional versions.
vboxuser@Ubuntu:~$ sudo apt install tree
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  libllvm19
Use 'sudo apt autoremove' to remove it.
The following NEW packages will be installed:
  tree
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 47.4 kB of archives.
After this operation, 111 kB of additional disk space will be used.
Get:1 http://es.archive.ubuntu.com/ubuntu noble-updates/universe amd64 tree amd64 2.1.1-2ubuntu3.24.04.
2 [47.4 kB]
Fetched 47.4 kB in 1s (38.5 kB/s)
Selecting previously unselected package tree.
(Reading database ... 256524 files and directories currently installed.)
Preparing to unpack .../tree_2.1.1-2ubuntu3.24.04.2_amd64.deb ...
Unpacking tree (2.1.1-2ubuntu3.24.04.2) ...
Setting up tree (2.1.1-2ubuntu3.24.04.2) ...
Processing triggers for man-db (2.12.0-4build2) ...
vboxuser@Ubuntu:~$ tree /var/www/proyecto
/var/www/proyecto
├── config
│   └── db_config.php
├── html
│   └── index.html
├── logs
│   └── access.log
├── media
└── scripts
    └── backup.sh

6 directories, 4 files
vboxuser@Ubuntu:~$
```

Paso 3 - Asignar propiertarios y grupos a cada directorio

Ahora asignamos los propietarios y grupos correctos usando chown. El propietario de todos los directorios será root y el grupo variará según el acceso que queramos delegar.

```
# Propietaro root, grupo desarrolladores para el directorio raiz
$ sudo chown root:desarrolladores /var/www/proyecto

# html: propietario root, grupo developers:
$ sudo chown root:desarrolladores /var/www/proyecto/html
$ sudo chown root:desarrolladores /var/www/proyecto/html/index.html

# config: solo root (sin delegar a ningún grupo):
$ sudo chown root:root /var/www/proyecto/config
$ sudo chown root:root /var/www/proyecto/config/db_config.php

# logs: propietario root, grupo auditores:
$ sudo chown root:auditores /var/www/proyecto/logs
$ sudo chown root:auditores /var/www/proyecto/logs/access.log

# scripts: propietario root, grupo developers:
$ sudo chown root:desarrolladores /var/www/proyecto/scripts
$ sudo chown root:desarrolladores /var/www/proyecto/scripts/backup.sh

# media: propietario root, grupo developers:
$ sudo chown root:desarrolladores /var/www/proyecto/media

# Verificar la asignación:
$ ls -la /var/www/proyecto/
```



Paso 4 - Asignar los permisos chmod a cada directorio

Ahora aplicamos los permisos exactos a cada directorio, usaremos la notacion octal (numérica) para ser precisos.

```
# Directorio raíz: rwxrwxr-x (775)
$ sudo chmod 775 /var/www/proyecto

# html/: rwxrwsr-x (2775) - con SetGID para herencia de grupo
$ sudo chmod 2775 /var/www/proyecto/html

# config/: rwx------ (700) - solo root
$ sudo chmod 700 /var/www/proyecto/config

# logs/: rwxr-x--- (750) - auditores pueden leer, others sin acceso
$ sudo chmod 750 /var/www/proyecto/logs

# scripts/: rwxr-x--- (750) - desarrolladores ejecutan, others sin acceso
$ sudo chmod 750 /var/www/proyecto/scripts

# media/: rwxrwxrwt (1777) - todos pueden escribir, sticky bit
$ sudo chmod 1777 /var/www/proyecto/media

# Asignar permisos a los ficheros individuales:
$ sudo chmod 664 /var/www/proyecto/html/index.html
$ sudo chmod 600 /var/www/proyecto/config/db_config.php
$ sudo chmod 640 /var/www/proyecto/logs/access.log
$ sudo chmod 750 /var/www/proyecto/scripts/backup.sh
```

```
# Verificar todos los permisos de una vez:

$ ls -la /var/www/proyecto/

$ stat /var/www/proyecto/html

$ stat /var/www/proyecto/config
```

Ubuntu (Instantánea 1) [Corriendo] - Oracle VirtualBox

Archivo Máquina Ver Entrada Dispositivos Ayuda

Feb 14 17:51

vboxuser@Ubuntu: ~

```
vboxuser@Ubuntu:~$ sudo chmod 775 /var/www/proyecto/
[sudo] password for vboxuser:
vboxuser@Ubuntu:~$ sudo chmod 2775 /var/www/proyecto/html/
vboxuser@Ubuntu:~$ sudo chmod 700 /var/www/proyecto/config/
vboxuser@Ubuntu:~$ sudo chmod 750 /var/www/proyecto/logs/
vboxuser@Ubuntu:~$ sudo chmod 750 /var/www/proyecto/scripts/
vboxuser@Ubuntu:~$ sudo chmod 1777 /var/www//proyecto/media/
vboxuser@Ubuntu:~$ sudo chmod 664 /var/www/proyecto/html/index.html
vboxuser@Ubuntu:~$ sudo chmod 600 /var/www/proyecto/config/db_config.php
vboxuser@Ubuntu:~$ sudo chmod 640 /var/www/proyecto/logs/acces.log
chmod: cannot access '/var/www/proyecto/logs/acces.log': No such file or directory
vboxuser@Ubuntu:~$ sudo chmod 640 /var/www/proyecto/logs/access.log
vboxuser@Ubuntu:~$ sudo chmod 750 /var/www/proyecto/scripts/backup.sh
vboxuser@Ubuntu:~$ ls -la /var/www/proyecto/
total 28
drwxrwxr-x 7 root desarroladores 4096 Feb 14 17:04 .
drwxr-xr-x 3 root root           4096 Feb 14 17:01 ..
drwx------ 2 root root           4096 Feb 14 17:04 config
drwxrwsr-x 2 root desarroladores 4096 Feb 14 17:04 html
drwxr-x--- 2 root auditores      4096 Feb 14 17:05 logs
drwxrwxrwt 2 root desarroladores 4096 Feb 14 17:04 media
drwxr-x--- 2 root desarroladores 4096 Feb 14 17:05 scripts
vboxuser@Ubuntu:~$ stat /var/www/proyecto/html
  File: /var/www/proyecto/html
  Size: 4096            Blocks: 8          IO Block: 4096   directory
Device: 8,2     Inode: 141179      Links: 2
Access: (2775/drwxrwsr-x)  Uid: (    0/    root)   Gid: ( 1001/desarroladores)
Access: 2026-02-14 17:48:59.527426016 +0000
Modify: 2026-02-14 17:04:43.646556547 +0000
Change: 2026-02-14 17:46:16.942192024 +0000
 Birth: 2026-02-14 17:03:58.838162999 +0000
vboxuser@Ubuntu:~$ stat /var/www/proyecto/config
  File: /var/www/proyecto/config
  Size: 4096            Blocks: 8          IO Block: 4096   directory
Device: 8,2     Inode: 149283      Links: 2
Access: (0700/drwx------)  Uid: (    0/    root)   Gid: (    0/    root)
Access: 2026-02-14 17:23:15.663300637 +0000
Modify: 2026-02-14 17:04:54.135798673 +0000
Change: 2026-02-14 17:46:40.625192313 +0000
 Birth: 2026-02-14 17:04:19.693585752 +0000
vboxuser@Ubuntu:~$
```

Paso 5 - Consigurar ACL para el usuario de veronica

El usuario veronica solo debe poder escribir en la carpeta media (para subir imágenes) sin pertenecer al grupo desarrolladores, aqui es donde las ACLs son imprescindibles

```
# Asegurarse de que las ACL están disponibles:

$ sudo apt install acl -y


# Conceder a veronica permisos de lectura y escritura SOLO en media/:

$ sudo setfacl -m u:veronica:rwx /var/www/proyecto/media


# Establecer ACL por defecto para que los nuevos ficheros hereden los permisos:

$ sudo setfacl -d -m u:veronica:rwx /var/www/proyecto/media


# Verificar que la ACL se ha aplicado correctamente:

$ getfacl /var/www/proyecto/media
```

Ubuntu (Instantánea 1) [Corriendo] - Oracle VirtualBox

Archivo Máquina Ver Entrada Dispositivos Ayuda

Feb 14 17:58

vboxuser@Ubuntu: ~

```
vboxuser@Ubuntu:~$ sudo apt install acl -y
[sudo] password for vboxuser:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
acl is already the newest version (2.3.2-1build1.1).
acl set to manually installed.
The following package was automatically installed and is no longer required:
  libllvm19
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 71 not upgraded.
vboxuser@Ubuntu:~$ sudo setfacl -m u:veronicarwx /var/www/proyecto/media
setfacl: Option -m: Invalid argument near character 3
vboxuser@Ubuntu:~$ sudo setfacl -m u:veronica:rwx /var/www/proyecto/media
vboxuser@Ubuntu:~$ sudo setfacl -d -m u:veronica:rwx /var/www/proyecto/media
vboxuser@Ubuntu:~$ getfacl /var/www/proyecto/media
getfacl: Removing leading '/' from absolute path names
# file: var/www/proyecto/media
# owner: root
# group: desarroladores
# flags: --t
user::rwx
user:veronica:rwx
group::rwx
mask::rwx
other::rwx
default:user::rwx
default:user:veronica:rwx
default:group::rwx
default:mask::rwx
default:other::rwx

vboxuser@Ubuntu:~$
```

Paso 6 - Configurar sudo para los desarrolladores

Configruamos sudo para que los desarrolladores puedan ejecutar el script de backup sin necesidad de ser root ni conocer la contraseña de root

```
# Editar el fichero sudoers de forma segura:

$ sudo visudo


# Añadir al final del fichero las siguientes líneas:

# (dentro del editor visudo)


# Permitir al grupo desarrolladores ejecutar el script de backup sin
contraseña:

%desarroladores ALL=(root) NOPASSWD:
/var/www/proyecto/scripts/backup.sh


# El auditor1 puede leer logs del sistema con sudo:

auditor1 ALL=(root) /usr/bin/tail, /usr/bin/grep
```

Ubuntu (Instantánea 1) [Corriendo] - Oracle VirtualBox

Archivo Máquina Ver Entrada Dispositivos Ayuda

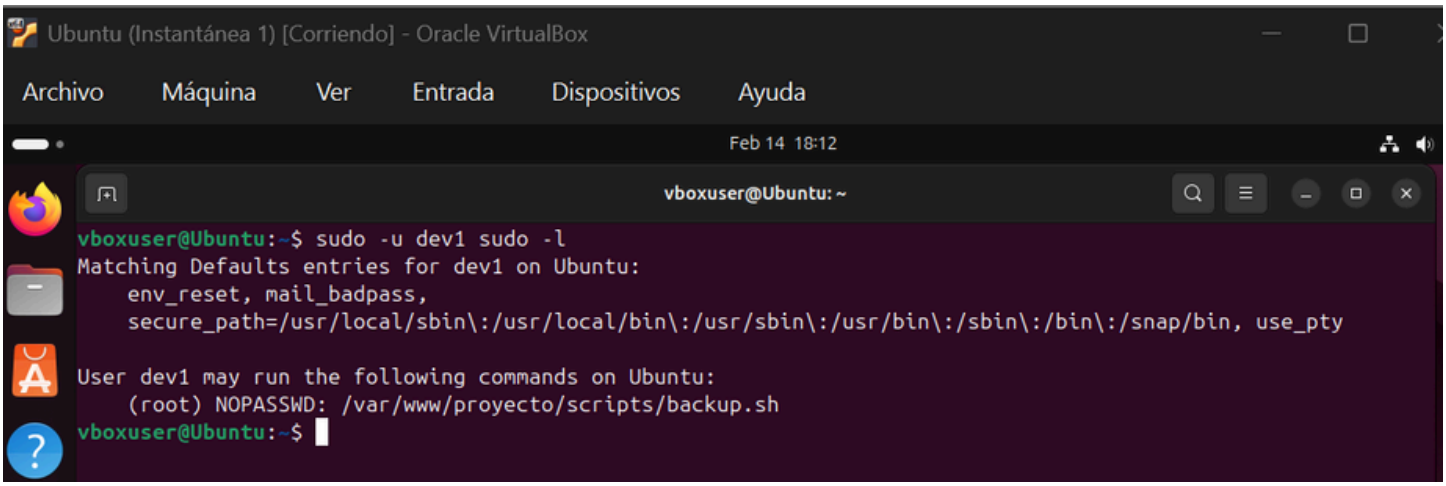
Feb 14 18:10

vboxuser@Ubuntu: ~

```
GNU nano 7.2                                  /etc/sudoers.tmp *
#Defaults:%sudo env_keep += "EDITOR"

# Completely harmless preservation of a user preference.
#Defaults:%sudo env_keep += "GREP_COLOR"

# While you shouldn't normally run git as root, you need to with etckeeper
#Defaults:%sudo env_keep += "GIT_AUTHOR_* GIT_COMMITTER_*"

# Per-user preferences; root won't have sensible values for them.
#Defaults:%sudo env_keep += "EMAIL DEBEMAIL DEBFULLNAME"

# "sudo scp" or "sudo rsync" should be able to use your SSH agent.
#Defaults:%sudo env_keep += "SSH_AGENT_PID SSH_AUTH_SOCK"

# Ditto for GPG agent
#Defaults:%sudo env_keep += "GPG_AGENT_INFO"

# Host alias specification

# User alias specification

# Cmnd alias specification

# User privilege specification
root    ALL=(ALL:ALL) ALL

# Members of the admin group may gain root privileges
%admin ALL=(ALL) ALL

# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL

# See sudoers(5) for more information on "@include" directives:

@includedir /etc/sudoers.d

# Permitir al grupo desarroladores ejecutar el script de backup sin contraseña:
%desarroladores ALL=(root) NOPASSWD: /var/www/proyecto/scripts/backup.sh

# El auditor1 puede leer logs del sietema con sudo:
auditor1 ALL=(root) /usr/bin/tail,/usr/bin/grep

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

 # Verificar que la configuración de sudo funciona para dev1:

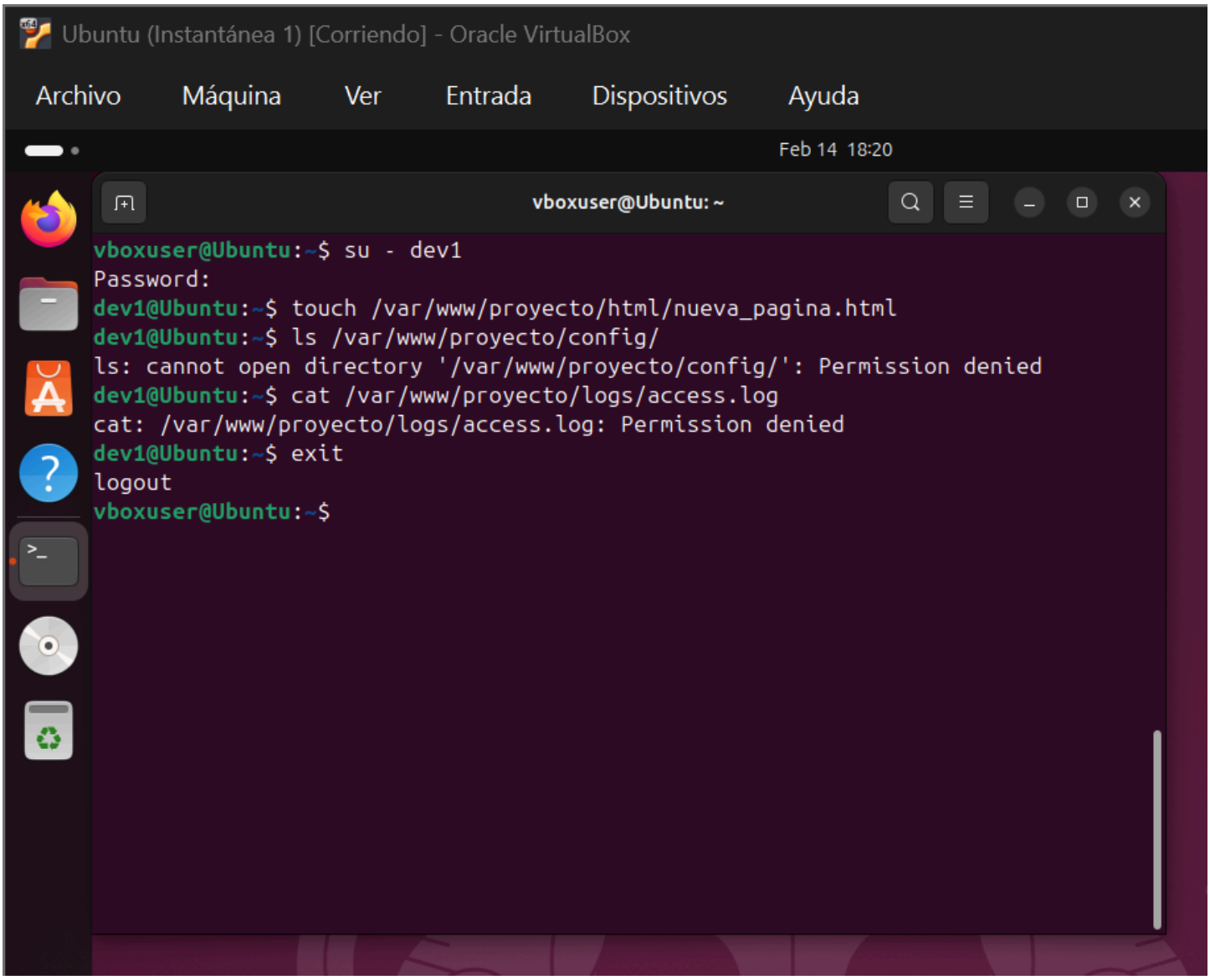
$ sudo -u dev1 sudo -l



Paso 7 - Verificación completa de los permisos efectivos

Para verificar que la configuración es correcta, probamos el acceso con cada usuario usnado el comando ***su*** (swich user)

```
#  Verificar acceso de dev1 (miembro de developers)

# Cambiar a usuario dev1:

$ su - dev1


# Intentar crear un fichero en html/ (debe funcionar):

$ touch /var/www/proyecto/html/nueva_pagina.html

$ echo 'OK'


# Intentar acceder a config/ (debe fallar - Permiso denegado):

$ ls /var/www/proyecto/config/

ls: cannot open directory '/var/www/proyecto/config/': Permission denied


# Intentar leer los logs (debe fallar - Permiso denegado):

$ cat /var/www/proyecto/logs/access.log

cat: /var/www/proyecto/logs/access.log: Permission denied


# Volver a root:

$ exit
```



Todo correcto! confirmamos que los permisos que hemos dado funcionan correctamente.